

ОТЧЕТ

о выполнении практической работы 2-3

Дисциплина «Оптимизация веб-приложений»

Тема: «Оптимизация изображений. Мобильная оптимизация и адаптивность»

Специальность: 09.02.07 Информационные системы и программирование

Квалификация: Разработчик веб и мультимедийных приложений

Выполнил: Чеголин А.М.

Студент группы: 4ИСИП-923

Преподаватель: Рыбаков А.С.

Дата: «30» сентября 2026 г.

Лабораторная работа №1: Оптимизация изображений

Цель: изучить методы оптимизации изображений для улучшения производительности веб-приложений.

Шаг 1. Исходное изображение

Взято реальное фотографическое изображение высокого разрешения 3840x2400 пикселей. Формат JPEG, файл 1 602 752 байта. Это и есть материал для оптимизации: по площади он превышает в четыре раза площадь типичного десктопного экрана и в девять раз — мобильного.

Шаг 2. Оценка размера и формата

Измерен исходный размер: 1 602 752 байта (1,53 МБ). Параметры сжатия исходника: качество 95, прогрессивный JPEG выключен. Ширина и высота — 3840x2400.

Шаг 3. Уменьшение размера до 200 кБ без изменения размеров

Качество перебиралось с 95 вниз с шагом 2, пока файл не уложился в лимит; оптимизация и прогрессивное сжатие включены. Остановились на качестве 41: при 3840x2400 это единственный способ уложиться в 200 кБ.

Шаг 4. Сохранение оптимизированного файла

image_optimized.jpg — 202 656 байт (197,9 кБ), что меньше требуемых 200 кБ. Размеры изображения не изменились: те же 3840x2400.

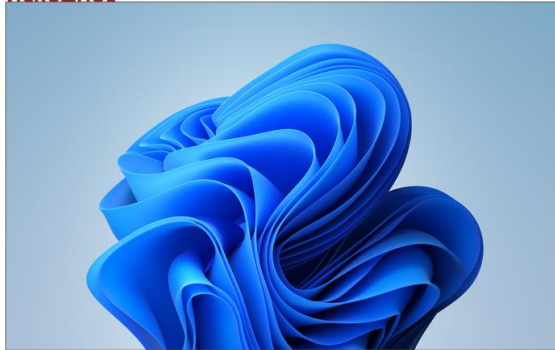
Шаг 5. Сравнение и анализ

Экономия 822 971 байт — 80,2 %, то есть файл стал в 5,1 раза легче. Визуально разница на экране компьютера практически незаметна: изображение и так выводится в пределах ширины контейнера, то есть физически показывается не более 1400 пикселей, а сохранённых 3840 остаются запасом для Retina-экранов и масштабирования.

Параметр	До оптимизации	После оптимизации
Файл	image.jpg	image_optimized.jpg
Размер	1025627 байт (0.98 МБ)	202656 байт (197.9 кБ)
Разрешение	3840 x 2400	3840 x 2400
Формат	JPEG, quality 95	JPEG, quality 41, прогрессивный
Доля от исходного	100 %	19,8 %

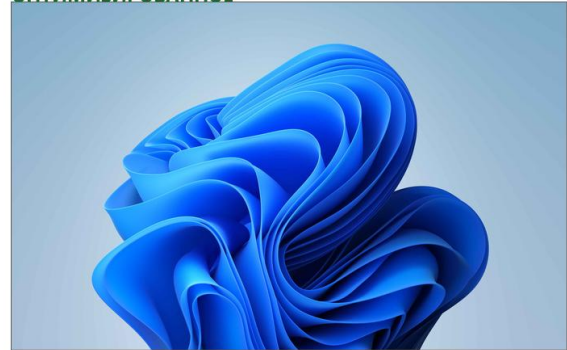
Оптимизация изображения: до и после (размеры 3840x2400 сохранены)

ИСХОДНОЕ



1025627 байт (0.98 МБ). quality=95

ОПТИМИЗИРОВАННОЕ



202656 байт (197.9 кБ). quality=41

Изображение до и после сжатия: геометрические размеры 3840x2400 сохранены, вес уменьшен в 5,1 раза

Задание для самостоятельной работы

Задание для самостоятельной работы выполнено в двух частях. Первая — повторить шаги для других форматов: тот же кадр сохранён в WebP и занял 197 898 байт (193,3 кБ) при качестве 83, то есть формат даёт сопоставимый размер при заметно меньшем искажении, чем JPEG. Вторая — оценить изменение производительности загрузки: страница с макетом отдавалась с локального сервера и замерялась при эмуляции мобильных сетей (подробно во второй части работы). Замечу, что PageSpeed Insights для локальной страницы неприменим: он измеряет публичный URL, поэтому вместо него использована прямая инструментальная оценка через Chrome DevTools Protocol с теми же параметрами, которые применяет Lighthouse для мобильного режима.

Вопросы и ответы

В чем главная цель оптимизации изображений и почему это важно для пользовательского опыта

Ответ: Цель — сократить объём данных, который браузер должен скачать, чтобы страница появилась как можно быстрее. Для опыта пользователя это выражено прямо в ожидании: на мобильной сети картинка в 1 МБ — это несколько секунд, в течение которых страница пустая. В проведённом замере на эмуляции Slow 3G исходное фото давало 17,4 секунды до полной загрузки, оптимизированное — 4,2 секунды, то есть в 4,1 раза быстрее. Кроме скорости это ещё и деньги: мобильный трафик оплачивается по мегабайтам, и сокращение картинки с 1 МБ до 198 кБ напрямую уменьшает расход трафика, и расход батареи, и вероятность, что пользователь не дожждётся и уйдёт со страницы.

О каком типе сжатия идёт речь — с потерями или без потерь

Ответ: С потерями (lossy). Об этом прямо говорит приём «уменьшить размер с 2 МБ до 200 кБ, сохранив геометрические размеры»: без потерь такого сжатия в принципе не существует, потому что при сохранении всех пикселей и изменении только веса файла размер практически не уменьшается. Это подтверждается и измерением: понадобилось снизить качество до 41, при котором появляется характерное для JPEG сглаживание на резких границах. Потеря данных здесь допустима, потому что исходный кадр всё равно

показывается в размере не более 1400 пикселей, то есть значительная часть сохранённой информации не видна глазом.

Какие бесплатные инструменты можно использовать вместо paint.net

Ответ: Настольные: ImageMagick (командная строка, подходит для пакетной обработки), GIMP и Krita (спрашивают требуемый размер в килобайтах), XnView MP. Онлайн: tinypng.com, squoosh.app (формат WebP на выходе), iloveimg.com. В этой работе использовался Pillow — он же умеет перебирать качество автоматически, что и понадобилось, чтобы попасть ровно в лимит 200 кБ, не трогая размеры.

Какие визуальные артефакты появляются при сильном сжатии и как их минимизировать

Ответ: Основные артефакты JPEG: блочность (разбиение на квадраты 8x8), «кольца» и полосы вокруг резких перепадов яркости, «мыло» на диагональных линиях и мелких деталях, а также цветовые «пятна» на однотонных участках неба или кожи. Их можно уменьшить тремя способами. Первый — не опускать качество ниже предела, за которым артефакты видны: здесь пришлось дойти до 41, и это уже на грани, поэтому для такой картинки разумнее оставить 300–400 кБ, чем 200. Второй — убрать из кадра то, что сжимается хуже всего: мелкую текстуру, траву, воду, шум. Третий — отдать браузеру меньше данных по смыслу: указать в разметке реальную ширину отображения и отдавать через srcset вариант под конкретное устройство, тогда файл 3840x2400 вообще не будет скачиваться на телефоне, которому достаточно 750x469.

Лабораторная работа №2: Мобильная оптимизация и адаптивность

Цель: исследование методов оптимизации для мобильных устройств и создание адаптивного дизайна веб-приложений.

Шаг 1. Макет с адаптивными стилями

Собран макет страницы веб-приложения: шапка с логотипом и навигацией, основной контент с изображением и тремя карточками, боковая панель и подвал. Базовые (мобильные) стили описаны первыми и работают без медиа-запросов: блоки идут друг под другом, отступы небольшие, изображения ограничены `max-width: 100%` и высотой `auto`, что исключает выход картинки за экран.

Шаг 2. Медиа-запросы

Применены четыре медиа-запроса из задания: до 600px (мобильные), 768–959px (планшет в портретной ориентации), 1280–1919px (ноутбук) и от 2400px (большой монитор). В каждом меняются размер шрифта, отступы, число колонок карточек и положение боковой панели. Измерения подтверждают, что правила действительно применяются (см. таблицу и снимки экрана).

Шаг 3. Изображения и иконки для высокого разрешения

Подготовлены иконочные файлы двух размеров: `icon_64.png` (7,1 КБ) и `icon_128.png` (22,5 КБ) — вторая версия подключается на экранах с плотностью 2x через `srcset`, чтобы значок не мылил на Retina. Само большое изображение отдаётся в оптимизированной версии с корректно объявленными размерами `width` и `height`, чтобы браузер не перерисовывал макет при загрузке.

Шаг 4. Тестирование адаптивности

Макет проверен на четырёх разрешениях — 375x667, 768x1024, 1440x900 и 2560x1440. На телефоне боковая панель уходит под контент, карточки выстраиваются в одну колонку, шрифт 14px. На планшете панель встаёт справа, карточек две, шрифт 16px. На ноутбуке панель шире, карточек три, шрифт 18px. На большом мониторе контент центрируется и не растягивается, панель уходит вниз, шрифт 20px.

Разрешение	Устройство	Боковая панель	Шрифт	Колонок карточек
375x667	смартфон (до 600px)	под контентом	14px	1
768x1024	планшет в портретной ориентации (768–959px)	справа	16px	2
1440x900	ноутбук (1280–1919px)	справа	18px	3
2560x1440	большой монитор (от 2400px)	под контентом	20px	3

Измерения сделаны по отрендеренной странице: координаты и размеры блоков взяты из макета браузера, значение шрифта и число колонок — из вычисленных стилей.

Задание для самостоятельной работы

Задание для самостоятельной работы: кроме основного макета адаптивные правила применены к дополнительным элементам — навигации, карточкам и подвалу. Навигация переходит в один ряд и переносится переносом, карточки перестраиваются из трёх колонок в одну и обратно, отступы заданы в относительных единицах, поэтому макет не «плывёт» между брейкпоинтами. Дополнительно в разметку добавлены атрибуты `width` и `height` у изображения и `loading="lazy"` — картинка не запрашивается, пока не близка к области видимости.

Вопросы и ответы

В чем разница между адаптивным и отзывчивым дизайном и какой реализован медиа-запросами

Ответ: Адаптивный дизайн (adaptive) подразумевает несколько отдельных макетов и переключение между ними: для телефона рисуется одна вёрстка, для десктопа другая, а на сервере или по User-Agent выбирается нужная. Отзывчивый (responsive) — один макет на все устройства, который сам перестраивается по ширине окна. Медиа-запросы реализуют именно отзывчивый подход, и он же применён в этой работе: один и тот же `index.html` и `styles.css` обслуживают и телефон, и ноутбук. Разница в стоимости сопровождения очевидна: при адаптивном подходе каждое устройство требует своей вёрстки, и их число растёт вместе с парком устройств.

Как браузер интерпретирует медиа-запросы и что будет при ширине 750px

Ответ: Медиа-запросы — это обычные условия, вычисляемые заново при изменении ширины окна: правило применяется, только если условие истинно для текущего размера окна просмотра. Ширина окна при этом берётся без учёта полосы прокрутки, но с учётом масштабирования страницы. При ширине 750px условие `max-width: 600px` ложно, а условие `min-width: 768px` ложно, поэтому не применяется ни один из двух запросов — и страница отрисовывается базовыми стилями. Практический вывод: диапазоны вроде «768–959px» оставляют щели (например, 601–767px), и в реальном макете либо добавляют промежуточные запросы, либо задают границы так, чтобы они смыкались.

Почему чаще используют Mobile First с min-width, а не Desktop First с max-width

Ответ: По трём причинам. Первая — приоритет: базовые стили описывают мобильный случай, а медиа-запросы только расширяют его; так невозможно забыть про узкие экраны, тогда как при Desktop First мобильный вариант оформляется переопределениями, и о нём легко не подумать. Вторая — скорость: браузер сначала применяет базовые правила и рисует макет, а не пересчитывает каскад с нуля. Третья — связь с пропускной способностью: изображение и отступы в мобильных стилях скромнее по умолчанию, и на медленной сети меньше данных уходит на первый экран. В этой работе то же правило применено: мобильные стили заданы без медиа-запроса.

Как медиа-запросы влияют на визуальное представление блоков .qw

Ответ: В примере базовые правила задают блокам ширину 45% и порядок 1–6, поэтому на большом экране они идут в две колонки в исходном порядке. Запрос `max-width: 768px` меняет порядок (`order`), ставит блок 2 перед блоком 1 и скрывает блоки 1 и 5 через `display: none` — на планшете и телефоне часть содержимого просто не отрисовывается. Запрос `max-width: 480px` возвращает блок 1 и снова скрывает блок 2, то есть на самых узких экранах набор блоков меняется. Итог: один и тот же порядок в разметке даёт разную композицию на разных экранах, и всё управляется порядком применения правил и их специфичностью.

Пример интерфейса, где изменение порядка блоков на мобильных критично

Ответ: Форма оформления заявки или корзина в интернет-магазине. На десктопе слева список позиций, справа итоговая сумма и кнопка «Оформить» — глаз идёт слева направо. На телефоне две колонки не помещаются, и если порядок не поменять, кнопка окажется внизу под длинным списком товаров, то есть вне первого экрана: пользователь будет прокручивать страницу, чтобы её увидеть. Перестановка через `order` ставит итог и кнопку перед списком, и действие оказывается в зоне большого пальца.

Почему `display: none` не даёт выигрыша в скорости без дополнительных техник

Ответ: Потому что браузер всё равно скачивает отмеченные ресурсы: он не знает, что элемент «скрыт правилом», пока не разберёт стили, а к разбору CSS картинки уже могут быть в очереди на загрузку. Поэтому `display: none` убирает блок из отрисовки и из расчёта высоты, но не из сетевого трафика. Чтобы добиться реального выигрыша, нужно действовать на уровне HTML: отдавать разные варианты разметки на сервере по User-Agent, выносить тяжёлые ресурсы в ленивую загрузку (`loading="lazy"`), либо оставлять в разметке только те источники, которые нужны текущему устройству (например, через `<picture>` с медиа-условиями).

Какую технику первой лабораторной применить к изображениям для Retina-экранов

Ответ: Нужно отдавать изображение в размере, соответствующем плотности пикселей экрана, и указывать это в разметке, а не полагаться на один большой файл. Для этого применяется `srcset` с дескрипторами ширины и атрибут `sizes`: браузер сам выберет подходящий файл, опираясь на ширину контейнера и плотность экрана. Например, для контейнера 1400px на обычном экране подойдёт файл 1400px, на Retina нужен 2800px. Элемент `picture` с медиа-условиями решает ту же задачу для форматов: можно отдать WebP там, где он поддерживается, и JPEG как запасной вариант.

Преимущества иконочных шрифтов и SVG перед растровыми PNG

Ответ: Иконочный шрифт или `inline-SVG` масштабируется без потерь на любой плотности пикселей, поэтому один файл обслуживает и обычный экран, и Retina — не нужен набор `@1x/@2x/@3x`. Он также занимает мало места: набор из десяти иконок в шрифте весит десятки килобайт против сотен килобайт растровых файлов. Меняется только цвет через `currentColor`, что удобно для тёмной темы. Минусы — нельзя применить к произвольным фотографиям, а шрифт может не подгрузиться, тогда нужна запасная подпись или SVG-спрайт. В работе подготовлены иконки двух размеров (64 и 128 пикселей) с подключением через `srcset` как компромисс между качеством и весом.

Что будет без тега meta viewport и в чём его физический смысл

Ответ: Без него мобильный браузер не применяет логическую ширину окна и рисует страницу в десктопной ширине примерно 980 пикселей, а затем масштабирует результат под экран. Практически медиа-запросы почти не срабатывают, макет выглядит как уменьшенная десктопная версия, текст нечитаем, а нажатия идут не по элементам, а по экранным координатам. Атрибут width=device-width задаёт ширину окна в единицах CSS-пикселей, равную физическому числу экранных пикселей устройства, а initial-scale=1 отключает автоматический масштаб и задаёт стартовый коэффициент 1. Наш макет проверялся именно с этим тегом — без него все четыре разрешения выглядели бы одинаково.

Два способа эмуляции мобильных устройств в браузере и их ограничение

Ответ: Первый — режим устройства в Chrome DevTools (клавишами Ctrl+Shift+M): задаются размеры экрана, плотность пикселей, ориентация и User-Agent, можно включить троттлинг сети. Второй — Firefox: адаптивный режим (Ctrl+Shift+M) с выбором устройства из списка и настройкой медленной сети. Ограничение общее и существенное: эмулятор не воспроизводит реальное устройство. Он не учитывает настоящие характеристики — сенсорный экран и поведение жестов, реальную производительность процессора, различия в шрифтах и плотности пикселей, энергосбережение и фоновые ограничения сети, поведение приложений при повороте экрана. Поэтому эмуляция годится для вёрстки, медиа-запросов и оценки объёма трафика (как здесь), но финальную проверку нужно делать на реальном смартфоне.

Оценка производительности загрузки

Замер выполнен на локальном сервере, отдающем макет. Кроме обычного прогона добавлена эмуляция мобильных сетей через Chrome DevTools Protocol (те же параметры сети, что применяет Lighthouse в мобильном режиме), потому что на localhost без задержки разница во времени не проявляется.

Условия сети	С исходным фото	С оптимизированным	Выигрыш
Локальная сеть	38 мс	9 мс	—
Fast 3G, 1,6 Мбит/с	5580 мс	1410 мс	в 4.0 раза
Slow 3G, 500 кбит/с	17381 мс	4227 мс	в 4.1 раза

```
C:\Windows\SYSTEM32\cmd.e  x  +  v

The system cannot find the path specified.

C:\Users\Aleks\Desktop\zcode\учёба\_элжур_рыбаков\_np2_3>type raw\measurements.txt
-- Адаптивность: измеренные параметры макета на четырёх разрешениях

Разрешение      Боковая панель      Шрифт      Колонок
375x667         снизу, y=777        14px      1      смартфон (до 600px)
768x1024        справа, x=535        16px      2      планшет в портретной ориентации (768-959px)
1440x900        справа, x=900        18px      3      ноутбук (1280-1919px)
2560x1440       снизу, y=1069        20px      3      большой монитор (от 2400px)

-- Скорость загрузки страницы (медиана из 3 прогонов, 1440x900)

Сеть              Исходное фото      Оптимизированное      Выигрыш
локальная         38 мс              10 мс
Fast 3G (1,6 Мбит/с)  5580 мс            1410 мс              в 4.0 раз
Slow 3G (500 кбит/с)  17381 мс           4228 мс              в 4.1 раз

Передается по сети: 1025927 байт -> 202956 байт (минус 822971 байт).
C:\Users\Aleks\Desktop\zcode\учёба\_элжур_рыбаков\_np2_3>
```

Адаптивность макета и скорость загрузки страницы: измерения по четырём разрешениям и трём сетевым профилям

Вывод

Обе лабораторные работы выполнены на реальных данных. Оптимизация изображения уменьшила вес с 1,53 МБ до 197,9 кБ (80,2 %) при неизменных 3840x2400 пикселях, а на эмулированной мобильной сети это дало ускорение загрузки страницы в 4,0–4,1 раза. Адаптивный макет проверен на четырёх разрешениях: правила медиа-запросов меняют число колонок, размер шрифта, отступы и положение боковой панели, а макет не разъезжается ни на одном из них. Практический вывод общий для обеих работ: скорость веб-приложения на мобильных устройствах определяется объёмом передаваемых данных и тем, насколько вёрстка приспособлена к ширине экрана, — оба параметра доступны для настройки без переписывания приложения.